

Joc laberint

Programació II
Miguel Mascaró

Jaume Adrover
Joan Balaguer

Vídeo explicatiu:
<https://youtu.be/lYOLVZSWeQ>

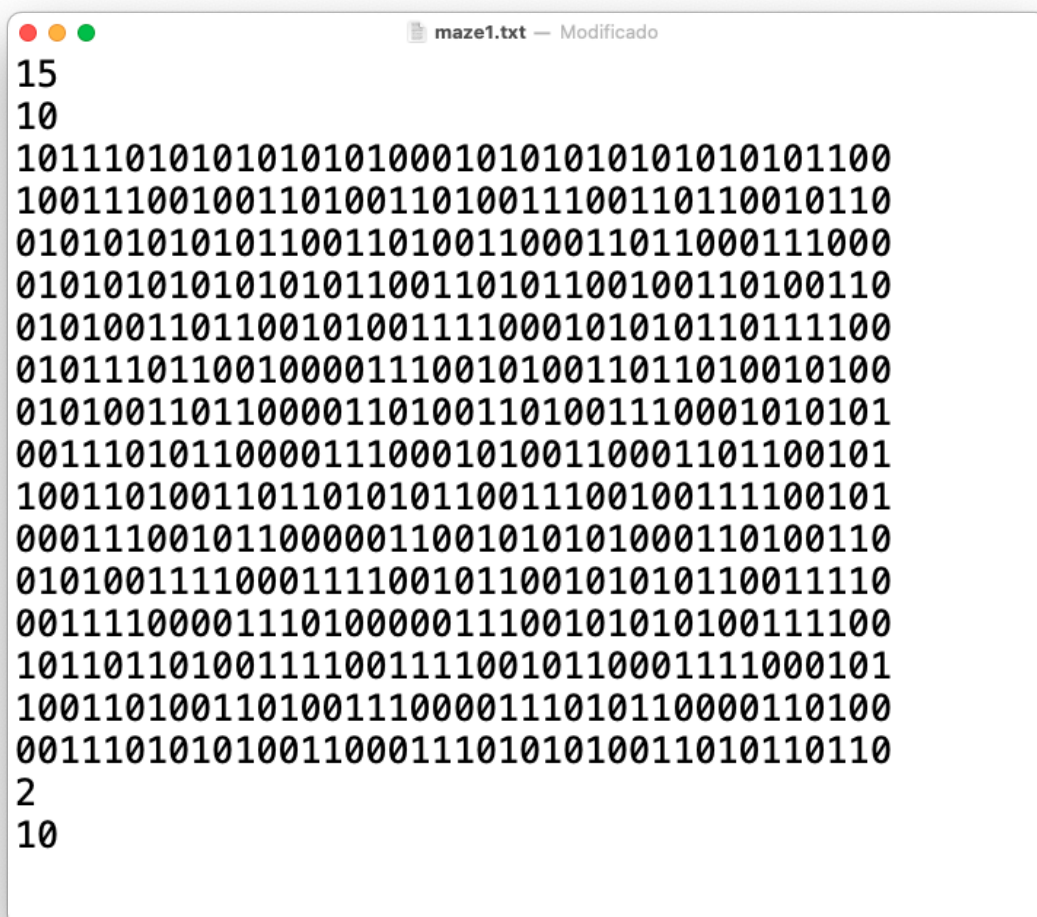
Índex

Introducció	3
Disseny	4
2.1 Classe Element	4
2.2 Classe Casella	4
2.3 Classe FitxerLaberints	5
2.4 Classe Laberints	5
2.5 Classe Main	6
Conclusions	7

1. Introducció

L'enunciat d'aquesta pràctica final ens demana l'implementació d'un laberint a partir d'una lectura de fitxers en binari. Les dues primeres línies del fitxer de text, ens defineixen el nombre de files i columnes del laberint, mentre que les dues últimes ens defineixen la coordenada on es troba la sortida. Les files del mig del fitxer, ens indiquen l'existència o no de les diferents parts de cada casella. En concret, si hi ha un 1, hi haurà una paret i si hi ha un 0 no. L'ordre d'aquests 1's i 0's no és aleatori, cada una de les caselles se li atribueixen 4 1's o 0's, depenent de la casella. Cada un d'aquests s'atribueix a una paret en concret de la casella: Nord, Est, Sud i Oest respectivament. L'usuari també ha de poder reiniciar el laberint, seleccionant el fitxer que vol que llegeixi el programa. Per últim, també ha de poder reiniciar la posició de la fitxa i sortir del programa.

Exemple format del fitxer que hem de llegir:



```
maze1.txt — Modificado
15
10
10111010101010101000101010101010101100
1001110010011010011010011100110110010110
0101010101011001101001100011011000111000
0101010101010101100110101100100110100110
0101001101100101001111000101010110111100
0101110110010000111001010011011010010100
0101001101100001101001101001110001010101
0011101011000011100010100110001101100101
1001101001101101010110011100100111100101
0001110010110000011001010101000110100110
0101001111000111100101100101010110011110
0011110000111010000011100101010100111100
1011011010011110011110010110001111000101
1001101001101001110000111010110000110100
0011101010100110001110101010011010110110
2
10
```

2. Disseny

El nostre programa utilitza un conjunt de classes i mètodes amb l'objectiu de garantir una correcta i senzilla estructura de dades.

Hem utilitzat les classes següents:

- Element
- Casella
- FitxerLaberints
- Laberint
- Main

2.1 Classe Element

La classe més senzilla del nostre projecte és aquesta, que inclou sols 3 atributs i 2 mètodes, el constructor i un `getImatge`. Ens representa l'existència d'un element que anirà introduït a una casella mitjançant un `drawImage`, que obtindrà per paràmetre la imatge d'un objecte `Element` instanciat.

2.2 Classe Casella

Casella és la classe a partir de la qual es construeix el laberint sencer. Aquesta classe `Casella` instancia objectes agafant les dimensions a partir d'un `Rectangle2D.float`. Aquesta classe conté els atributs següents: `rect` (`Rectangle2D.float`), `blnparets []` (array de 4 booleans, un objecte `Element` '*element*' i un boolean anomenat *plena*.

La seva instanciació es duu a terme mitjançant un array de 4 ints, que constituïran les parets i un rectangle, que vendrà a determinar les dimensions de la nostra casella. L'array de booleans `blnparets []` s'inicialitza en funció de si hi ha parets o no, que més tard controlaran si la fitxa pot moure-se.

Aquesta classe té els mètodes següents:

- Inserir: posa el boolean `plena` a `true`, per tal de realitzar la inserció d'una imatge a dins el `paintComponent`, que es definirà dedins aquest mateix mètode. És a dir, inserim una fitxa o la sortida.

- Retirar: posa el boolean *plena* a false per tal de que després d'un repaint() s'esborri l'existència de possibles elements a les caselles.
- getParets: mètode que ens retorna l'array de parets d'una casella per tal d'examinar un futur moviment de la fitxa i veure si és possible.
- paintComponent: mètode que ens dibuixa les caselles i els seus corresponents elements si el boolean *plena* es true;

2.3 Classe FitxerLaberints

FitxerLaberints està dedicada exclusivament a la lectura de fitxers de text per a construir el laberint. Tenim un atribut `BufferedReader lector`, un array `int [] parets` i dos ints `files`, `columnes`. Tenim els mètodes següents:

- Mètode constructor: inicialitza el `BufferedReader` amb l'enllaç del fitxer de text, donat per paràmetre
- Mètode LlegirDimensió: realitza la lectura de files i columnes.
- Mètode LlegirParets: realitza la lectura de parets, inserint-los dins un array.
- Mètode llegir: és equivalent a `x.read()` de la classe `BufferedReader`.
- Mètode llegirlinea: equivalent a `x.readLine()` de la classe `BufferedReader`.
- Mètode getFiles: retorna el nombre de files llegit.
- Mètode getColumnes: retorna el nombre de columnes llegit.

2.4 Classe Laberints

Classe general que engloba totes les altres, com a atributs de la mateixa. Té a dins atributs com la Dimensió X,Y ; la mida del costat, el lector `FitxerLaberints`, `files`, `columnes` i les coords de la fitxa i de la sortida respectivament, en dos arrays. Consisteix bàsicament en un array bidimensional de objectes `Casella`, que es van dibuixant en el `paintComponent` d'aquesta mateixa classe.

S'instancia a través d'un fitxer donat per paràmetre, el qual serà llegit i construirà l'estructura de caselles. Tenim diferents mètodes:

- Mètode `AssignarFilesColumnes`: assigna els valors a les files i columnes respectivament.
- Mètode `AssignarSortida`: assigna segons la lectura de les darreres 2 línies la posició de la sortida.

- **Mètode AssignarFitxa:** es genera aleatoriament la posició on sortirà la fitxa
- **Mètode moureFitxa:** mètode que rep per paràmetre un char c, que es la lletra que determina el moviment de la fitxa(a,w,s,d), oest,nord,sud i est respectivament.
- **Mètode checkVictoria:** comprova si les coords de la Fitxa i de la sortida coincideixen, cosa que ens diu que la fitxa ha arribat a la sortida.
- **mètode GetPreferredSize:** retorna la dimensió del laberint per a utilitzar-la en el JFrame principal.
- **Mètode paintComponent:** dibuixa el laberint i les seves respectives caselles mitjançant un doble bucle for.

2.5 Classe Main

Classe que hereda les característiques de JFrame i implementa un KeyListener, és a dir, un escoltador de teclat per tal de controlar el moviment de la fitxa mitjançant les tecles a,w,s,d. A les declaracions tenim: components del menú(reiniciar laberint, reiniciar fitxa i sortir), que s'implementen mitjançant un JMenuBar.

En el constructor definim totes les propietats del JFrame com per exemple el setResizable(false), setTitle("Joc laberint")... Afegirem també un add.KeyListener(this),per tal de captar el teclat. Finalment, cridam al mètode initComponents que realitza tota la execució i inicialitza totes les components del JFrame.

Mètodes:

- **InitComponents:** s'inicialitzen les components del menú i el mateix laberint.
- **TriarLaberintInicial:** s'elegeix aleatòriament un dels 4 laberints.
- **ReiniciarFitxa:** es fa un remove del laberint actual i es torna a afegir, ja que la fitxa surt a una posició aleatòria nova, sense haver canviat el laberint.
- **ReiniciarLaberint:** es reinicia el laberint mitjançant un JFileChooser.
- **KeyListener:**
 - **KeyPressed:** primer cridam al mètode moureFitxa, després es fa un repaint() i finalment es comprova si s'ha arribat a la sortida
 - **KeyReleased:** no utilitzat.
 - **KeyTyped:** no utilitzat.

3. Conclusions

L'elaboració d'aquesta pràctica ens ha estat de gran utilitat per a consolidar tots els coneixements adquirits aquest curs, tant per l'apartat gràfic, com per a reforçar les nostres capacitats algorítmiques. Sobretot, ha suposat un repte de cara a realitzar de manera correcte un disseny descendent com toca. Això, és degut a que, en un principi, no varem fer l'estructura de dades com tocava i això, ens va fer replantejar-mos la vertadera utilitat de cada classe. Trobam que el nostre professor ens va donar uns consells molt útils de cara a realitzar de manera correcte tant el disseny descendent com l'estructura de dades. Tot i aquest mal plantejament inicial, trobem que hem realitzat una bona feina, i estem orgullosos del treball fet.